Architecture A vs Architecture B

OCR → LLM invoice extraction: what the two architectures are, what was changed, how to run them, and what they achieved

Thesis: Latency and Accuracy Optimization for OCR to LLM Document Processing Pipelines · 20 October 2026 · Hardware: NVIDIA RTX 3050 Laptop GPU (4 GB), 12 CPU threads, 16 GB RAM

5.7×	**0.94 / 0.94**	**92.5 %**	**600**
throughput of B over A 53.1 → 301.7 invoices/min	F1 of A / B with the current code (was 0.85)	invoices accepted without review – 90.5 % fully correct	invoices evaluated 300 real, 300 synthetic

Contents 1. The task · 2. The two architectures · 3. What was changed · 4. How to run · 5. Results

1. The task

Input: one invoice image (PNG, JPG or WebP). Output: three fields – **invoice number**, **date** and **total**. Both architectures solve exactly this task with exactly the same components: Tesseract 5 for OCR, the MiniLM sentence-embedding model to pick the relevant lines, and Google's Flan-T5-base language model to read the fields. They differ only in how the work is organised. That is the point of the thesis: measure how much an optimised execution architecture saves in time, memory and cost, while keeping accuracy the same.

2. The two architectures

2.1 Architecture A – the sequential baseline

One invoice at a time. Each step waits for the previous one; the next invoice starts only when the current one is finished.

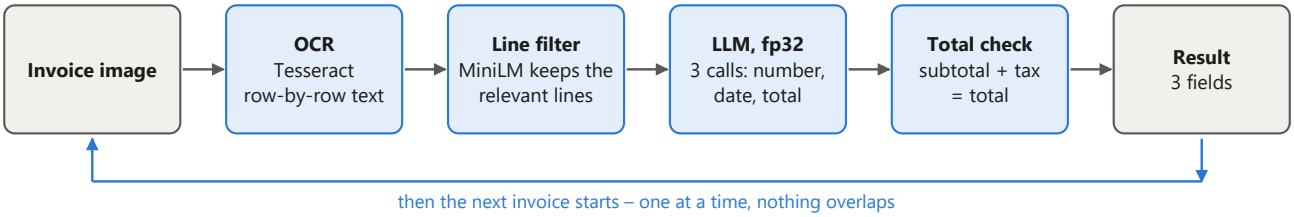


Figure 1 – Architecture A (code: `pipeline/arch_a.py`).

2.2 Architecture B – the optimised pipeline

All invoices are submitted together. Already-seen invoices come straight from the cache; the rest flow through a pipeline in which OCR and the GPU work at the same time on different invoices.

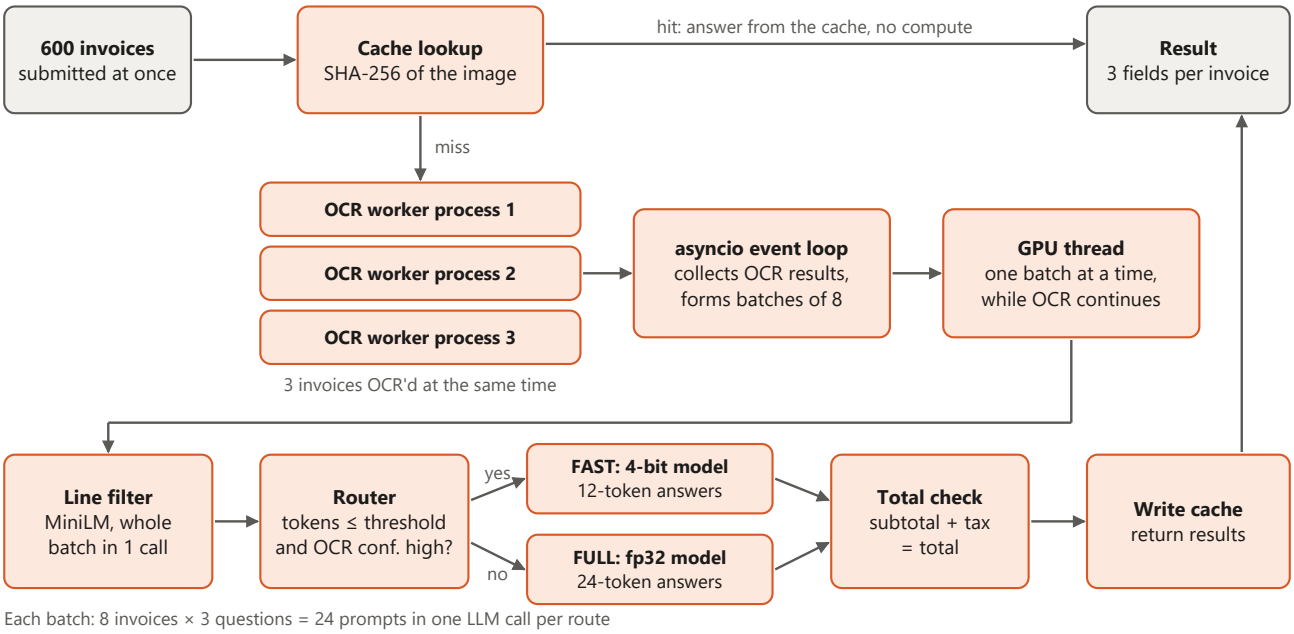


Figure 2 – Architecture B (code: `pipeline/arch_b.py`).

2.3 Side by side

	Architecture A	Architecture B
Order of work	One invoice at a time, start to finish	All stages busy at once, on different invoices
OCR	In the main process, one image at a time	3 worker processes, 3 images at a time
Line filter (MiniLM)	One call per invoice	One call per batch of 8 invoices
LLM calls	3 per invoice (one per field)	1 per batch and route: 8 invoices × 3 fields = 24 prompts
Model precision	fp32 only	Router chooses per invoice: 4-bit (FAST) or fp32 (FULL)
Repeated invoices	Processed again from scratch	Answered from a SQLite cache keyed by the image's SHA-256
Coordination	A plain loop	asyncio event loop: OCR in a process pool, GPU work in one thread
Same in both	Tesseract, MiniLM, Flan-T5-base, prompts, total check, scorer	

An analogy. A is one cook who takes an order, chops, cooks, plates, and only then takes the next order. B is a kitchen where three people chop at the same time while one cook prepares eight plates on one stove, and regular customers get yesterday's order from the fridge. The dishes are the same; the number of plates per hour is not.

Why B helps only on many invoices. On a single invoice there is nothing to overlap or batch, so A and B take the same time (1.32 s vs 1.31 s for invoice 318). B's advantage is throughput on a stream of invoices.

2.4 What the two architectures looked like when the code was received

The two-architecture design was already in the original code. The problem was that the two could not be compared fairly:

Original file	What it was	Problem
`baseline_pipeline.py`	Early pilot: OCR + keyword matching, no LLM	Not an AI pipeline (the supervisor asked for OCR → embeddings → LLM)
`baseline_llm_pipeline.py`	Architecture A: OCR → MiniLM → Flan-T5	Ran on the CPU; answers taken from a regex over the LLM's output
`main_optimized.py`	Architecture B: cache, routing, 8-bit Flan-T5	1 worker (effectively sequential); answers from a regex over the OCR text; no batching, no asyncio
`main_optimized_4bit.py`	Architecture B with 3 workers	Called "4-bit" but loaded the model in 8-bit
`accuracy_*.py` (3 scripts)	Separate scorers for A and B	Different rules (0.7 vs 0.9 similarity), only the first digits compared, wrong 50 invoices

All original files are kept, unchanged, in `legacy/`.

3. What was changed

3.1 Defects fixed (the reasons the earlier results were wrong)

#	Defect	Effect	Fix
1	Flan-T5 was asked for JSON, but has no { } tokens	LLM output never used; answers came from regex	Ask each field as a question
2	accuracy_check.py scored the wrapper dict	Every field empty; crash on cache hit	One scorer, scores the prediction
3	Only the first digit group was compared	$6,270 matched any $6,xxx	Amounts compared as numbers
4	A scored at 0.7 similarity, B at 0.9	Comparison invalid	One scorer, one threshold (≥ 90)
5	Files sorted as text	"First 50" = invoices 1, 10, 100–143	Numeric order, all 600
6	Invoice 1's label named test_invoice.json	Invoice 1 always skipped	Renamed
7	A ran on the CPU, B on the GPU	Speed-up was mostly CPU vs GPU	Both on the GPU
8	"4-bit" script loaded 8-bit	No 4-bit experiment existed	fp32 / fp16 / 8-bit / 4-bit all measured
9	Throughput from summed times; memory in parent only	Parallelism invisible; GPU memory unmeasured	Wall-clock time; process tree + GPU sampled 4×/s
10	Cache keyed by file name, read back with eval()	Stale results; code-execution risk	SHA-256 key + configuration; JSON
11	Fallback regex cut totals at the first separator	$12,345.67 read as 12,345	Removed with the regex fallback

3.2 Built to meet the supervisor's and committee's requirements

- **Architecture A** as OCR → embeddings → LLM, sequential, fp32 – the same components as B, on the GPU.
- **Architecture B** with asyncio + `run_in_executor`, a 3-process OCR pool, batches of 8, 4-bit and fp32 routes, and a content-addressed cache.
- **Routing** on token count and OCR confidence, with thresholds derived from data (grid search over all-4-bit vs all-fp32 runs) instead of guessed.
- **F1 scoring** (TP/FP/FN per field) with fuzzy matching ≥ 90 for invoice numbers and semantic comparison of dates and amounts.

- **All 600 invoices, 3 runs per configuration**, mean ± standard deviation; real and synthetic reported separately.
- **Correct metrics:** wall-clock throughput, service time vs latency, peak RAM and GPU memory, cost at €0.35 per GPU-hour.
- **Reproducibility:** hardware and versions recorded in every run; keep-awake during runs; README, requirements.txt, 55 unit tests.

3.3 Accuracy upgrade – shared by A and B

Error analysis showed where accuracy was lost: on real invoices only 32 % of totals were right. Two causes, two fixes – both standard in commercial invoice processing (AWS Textract, Azure Document Intelligence, Google Document AI):

Cause	Fix	Code
Tesseract reads tables column by column, so "Total" and "$ 327,08" ended up 30 lines apart	Rebuild the text row by row from each word's position on the page: `Total | $ 297,35 | $ 29,73 | $ 327,08`	`pipeline/ocr.py` `OCR_LAYOUT = "rows"`
The model takes the first amount after "Total" – the net, not the amount due	Arithmetic check: if amounts on the total line satisfy subtotal + tax = total, use that total; otherwise keep the model's answer	`pipeline/validate.py` `RECONCILE_TOTALS = True`

Measured on all 600 invoices with everything else unchanged: F1 0.847 → 0.859 with row text alone, **0.940** with both; totals on real invoices 32 % → 81 %. Of the 146 totals that changed from wrong to right, 121 are on labels written by the original author, so the gain does not depend on the label corrections below.

3.4 Data, environment and files

- **Ground truth:** `test_invoice.json` renamed to `test_invoice1.json`. 26 labels on invoices 316–600 held the net instead of the gross total and were corrected to the gross (the amount due), each verified by net + VAT = gross on the OCR output. Originals in `ground_truth_backup_20261018/`, list in `results/relabel_totals_log_pass1.csv` (25) and invoice 417. Images untouched.
- **Environment:** Tesseract 5.4 installed; new `.venv` (Python 3.12, PyTorch 2.5.1 + CUDA 12.1, bitsandbytes 0.50); the two broken virtual environments from another PC deleted.
- **New code:** `pipeline/` (OCR, filter, LLM, cache, routing, validation, metrics, A, B), `evaluation/` (scorer, benchmark runner, report, routing analysis, re-score, relabel), `tests/`, `demo.py`. No comments in the code; the explanations are in `docs/`.

4. How to run

Open PowerShell in `G:\Freelance\ocr_llm_project\ocr_llm_project`. On this machine everything is installed.

What	Command
Check the GPU	`.venv\Scripts\python -c "import torch;print(torch.cuda.get_device_name(0))"`
Unit tests (55)	`.venv\Scripts\python -m pytest tests -q`
Demo: A and B side by side	`.venv\Scripts\python demo.py --docs 30`
Demo on real invoices only	`.venv\Scripts\python demo.py --docs 100 --source real`
Quick benchmark (24 invoices)	`.venv\Scripts\python -m evaluation.run_experiments --configs A B --runs 1 --limit 24 --out results_check`
Full benchmark, A and B	`.venv\Scripts\python -m evaluation.run_experiments --configs A B --runs 3`
Every configuration (≈ 4 h)	`.venv\Scripts\python -m evaluation.run_experiments --configs all --runs 3 --skip-existing`
Tables and figures	`.venv\Scripts\python -m evaluation.report`
Routing thresholds from data	`.venv\Scripts\python -m evaluation.routing_analysis`
Re-score stored runs	`.venv\Scripts\python -m evaluation.rescore`

4.1 One invoice through both architectures

```
.venv\Scripts\python -c "
from pipeline.dataset import load_documents
from pipeline.arch_a import ArchitectureA
from pipeline.arch_b import ArchitectureB
docs = load_documents(indices=[318])
a = ArchitectureA().run(docs)[0]; print('A', a.prediction, a.truth)
b = ArchitectureB().run(docs)[0]; print('B', b.prediction, b.route)
"
```

4.2 Where the results go

- `results/summary.md` – all tables, mean ± standard deviation; `results/figures/` – 10 charts.
- `results/runs/<configuration>/run_<k>/documents.csv` – one row per invoice: timings, route, prediction, truth, outcome per field.
- `summary.json` next to it – metrics, scores, configuration, hardware and library versions of that run.

4.3 For valid timings

- Keep the laptop **plugged in and idle**. Using it during a run slows OCR (0.68 → 0.94 s per invoice was observed) and distorts the speed numbers; accuracy is not affected.
- If OCR time per invoice in `documents.csv` is well above 0.7 s, something else was using the CPU – repeat that run.
- Add `--offline` to stop the run from contacting Hugging Face once the models are downloaded.

5. Results

Final benchmark with the current code: 600 invoices, 12 configurations × 3 runs, laptop idle and plugged in, models loaded offline. Values are mean ± standard deviation. Runs disturbed by other programs were detected (OCR time per invoice) and repeated; they are kept in `discarded/` of the benchmark folder `outputs/2026-10-18_23-52-41_full_benchmark`.

5.1 Speed, memory and cost

Metric	Architecture A	Architecture B	B vs A
Wall time for 600 invoices	677.9 ± 7.4 s	119.4 ± 4.7 s	5.7× faster
Throughput	53.1 ± 0.6 /min	301.7 ± 11.8 /min	5.7×
Compute per invoice (service time)	1.12 s	0.72 s	1.6× lower
of which OCR / LLM	0.59 s / 0.51 s	0.58 s / 0.13 s	LLM 3.8× lower
Time per invoice, one at a time (p95 latency)	1.79 s	–	B is built for throughput
GPU memory used by the pipeline (peak)	1067 MB	1996 MB	1.9× higher
Peak RAM (all processes of the run)	1438 MB	2012 MB	+40 %
Cost per 100 invoices (€0.35/GPU-h)	€0.0110	€0.0019	5.8× lower
Same 600 invoices again (warm cache)	–	0.59 s	≈ 1 ms per invoice

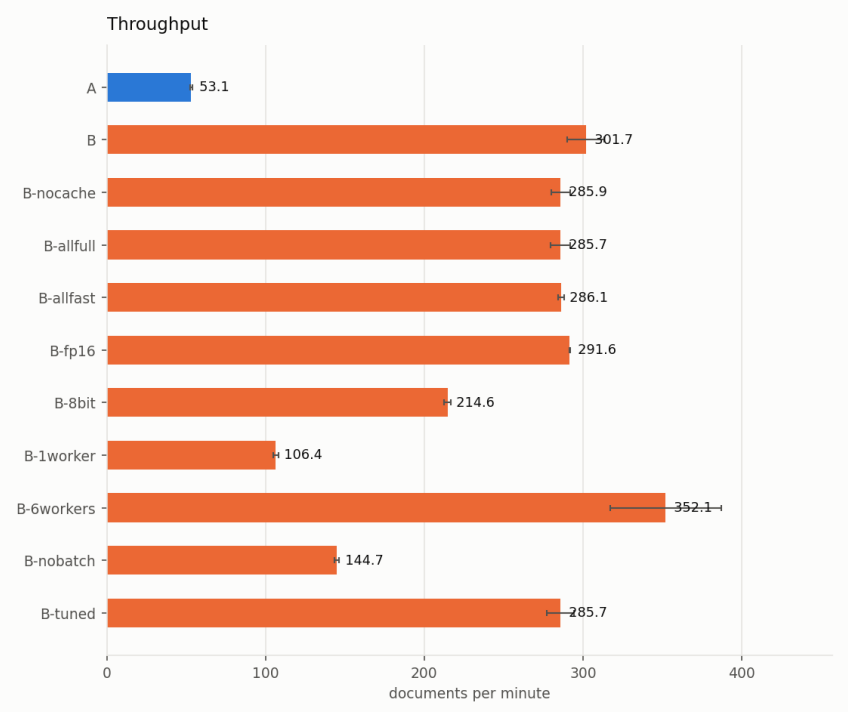


Figure 3 – Throughput of every configuration (mean ± standard deviation, 3 runs). A in blue, B and its variants in orange.

5.2 Where B's speed comes from

Each variant changes one part of B; everything else stays the same.

Configuration (one change to B)	Time for 600	Throughput	What it shows
B as designed (3 OCR workers, batch 8)	119.4 s	302 /min	reference
1 OCR worker instead of 3	338.4 s	106 /min	Parallel OCR is worth 2.8×
No batching (batch size 1)	248.8 s	145 /min	GPU batching is worth 2.1×
6 OCR workers	102.9 ± 10.3 s	352 /min	16 % faster again, but more RAM (2.6 GB) and less stable – the laptop's 12 threads start to saturate
No cache (first pass)	126.0 s	286 /min	No measurable cost on a first pass; repeats take 0.59 s instead of ~120 s

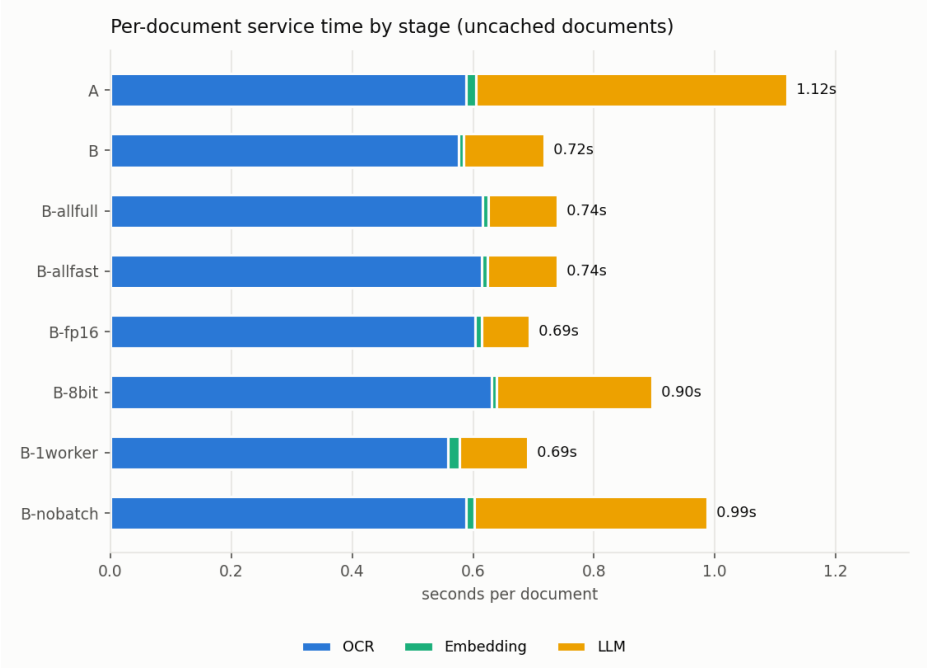


Figure 4 – Compute per invoice by stage. OCR costs the same everywhere; B saves in the LLM stage by batching. The rest of the 5.7× comes from OCR and GPU working at the same time, which this per-invoice view does not show.

5.3 Accuracy

F1 combines precision and recall over the three fields (1.0 = every field right). Same scorer and ground truth for every row.

Version of the code	Arch.	F1 all 600	F1 real (300)	F1 synthetic (300)	Real: number / date / total	Real invoices fully correct
Before the accuracy upgrade (benchmark, 3 runs)	A	0.847	0.704	0.989	0.93 / 0.86 / 0.32	24 %
	B	0.850	0.712	0.987	0.93 / 0.86 / 0.34	26 %
Current code: row text + total check (benchmark)	A	0.941	0.892	0.989	0.94 / 0.92 / 0.82	72 %
	B	0.941	0.894	0.987	0.94 / 0.92 / 0.83	72 %

- **A and B are equally accurate** (F1 0.941) – by design: they share the extraction. The tiny per-field differences come from B's FAST route; on the FULL route A and B give identical answers.
- **The accuracy upgrade is the big step**: F1 0.85 → 0.94 overall; on real invoices the total went from about a third right to 82–83 % right.
- Per field over all 600 invoices (B): invoice number 0.967, date 0.960, total 0.896. Synthetic invoices: 0.99; real invoices: 0.89 – reported separately for that reason.

5.4 Human review

Each invoice gets a review flag when a field is missing or unreadable, the total does not appear on a total line, or OCR confidence is below 80. Flagged invoices go to a person; the rest are accepted automatically.

	Architecture A	Architecture B
Accepted without review (straight-through)	92.0 %	92.5 %
Fully correct among accepted invoices	90.9 %	90.5 %
Fully correct if every invoice were accepted	84.2 %	84.2 %
Flagged invoices that really were wrong	94 % (45 of 48)	93 % (42 of 45)
Real invoices only: accepted / fully correct among accepted	87.3 % / 81.3 %	89.0 % / 80.5 %

Fewer than 1 in 12 invoices go to a person, and almost every one of them really needed checking. The flag cannot catch a wrong invoice number or date that looks valid – those remain the main residual errors.

5.5 Model precision and quantization

The same pipeline with every invoice on one model, loaded in four precisions:

Precision	Time for 600	LLM per invoice	Pipeline GPU peak	F1
fp32	126.1 s	0.115 s	1645 MB	0.941
fp16	123.5 s	0.079 s	1049 MB	0.941
4-bit NF4 (bitsandbytes)	125.8 s	0.115 s	1102 MB	0.941
8-bit LLM.int8 (bitsandbytes)	167.8 s	0.258 s	1176 MB	0.939 (varies slightly between runs)

- **4-bit saves memory, not time**: one third less GPU memory than fp32 at the same speed and accuracy.
- **fp16 is the best option on this GPU**: the fastest LLM step, the least memory and the same accuracy. For a model of 248 M parameters, quantization brings no advantage over half precision.
- **8-bit is the slowest** (2.2× slower LLM step than fp32) and its outlier handling makes results vary slightly from run to run.

5.6 Routing

- B as designed routes by text length and OCR confidence with initial thresholds (300 tokens, confidence 85): 505 invoices to the 4-bit model, 95 to fp32.
- **Thresholds derived from the data** (grid search over the all-4-bit and all-fp32 runs, accuracy within 0.5 percentage points of all-fp32): 525 tokens, confidence 70 – 99.8 % of invoices go to the 4-bit model. The two models disagree on only 10 invoices, 5 each way. Validated with three fresh runs (B-tuned): F1 0.941, 126.1 s.
- The router works; with two precisions of the same small model it has little to choose between. It becomes useful when the two routes differ more – e.g. a small model for simple invoices and a larger one for complex invoices.